Ticket To Ride

Design Document

Author:

Hew Jia Jing (33589739)

Document Revision History

Date	Version	Description
22/05/2026	0.0	Initial draft
22/05/2026	0.1	Add introduction
23/05/2026	0.2	Add UML class diagram and initial design rationale
24/05/2026	0.3	Improve design rationale
25/05/2026	0.4	Improve design rationale
28/05/2026	0.5	Improve design rationale
30/05/2026	0.6	Improve design rationale
31/05/2026	0.7	Improve design rationale

1 Introduction

This document provides an overview of the software architecture and design for the Ticket To Ride: London Edition game, allowing the development team to implement the system based on this document. The document outlines the overall system architecture, including the structure, refactoring techniques and rationale behind each key design decision.

This document is organised into the following sections:

- Section 2: Updated Class Diagram - Presents the structural design of the system focusing on Sprint 3 extensions, presented through UML class diagrams
- Section 3: Antipatterns and Refactoring - Discusses identified design antipatterns and the refactoring techniques applied
- Section 4: Design Rationales - Explains and justifies the major design decisions, including class responsibilities, system quality attributes and applied design patterns

Gitlab Repository Link:
https://git.infotech.monash.edu/FIT3077/fit3077-s1-2026/assignment-individual/jhew0010

Contributor Analytics:

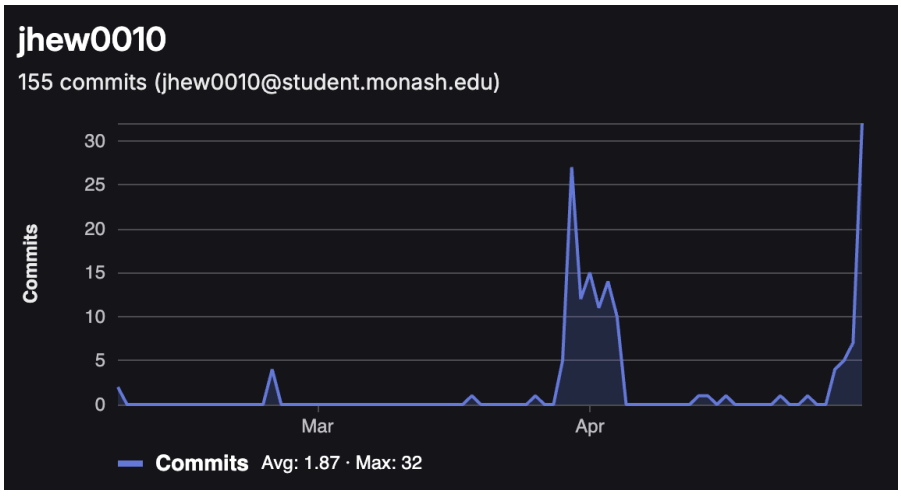


Figure 1: Contributor Analytics

2 Updated Class Diagram

I implemented three extensions for Sprint 3: Ferry Routes, Save/Load functionality and Self-Defined Player Abilities. These extensions introduce new gameplay logics and features to the Ticket To Ride game.

2.1 Ferry Routes

This extension introduces a new type of route that covers over bodies of water. Each ferry route can be identified by at least 1 locomotive symbol on the route. To successfully claim the route, players must pay a locomotive card for each locomotive symbol that is shown on the route, and corresponding normal train cards for the remaining space of the route. Players are able to use a special rule where if a player does not hold enough locomotive cards, every missing locomotive symbol can be replaced by 3 train cards of any colour.

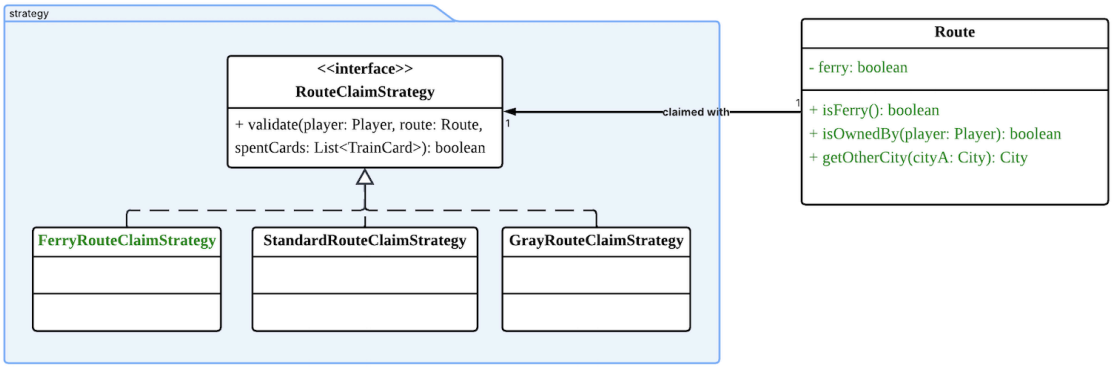


Figure 2: Ferry Route sub-UML class diagram

2.2 Save and Load Functionality

This extension allows players to save an ongoing game and resume playing it at a later time.

Save: Top right corner of the game

Load: Main Menu page of the game

The saved game stores all necessary information to resume the game session, including player progress, cards, decks, routes, scores, turn information and end-game state. However, to ensure fairness, games can only be saved before the current player has performed any action during their turn.

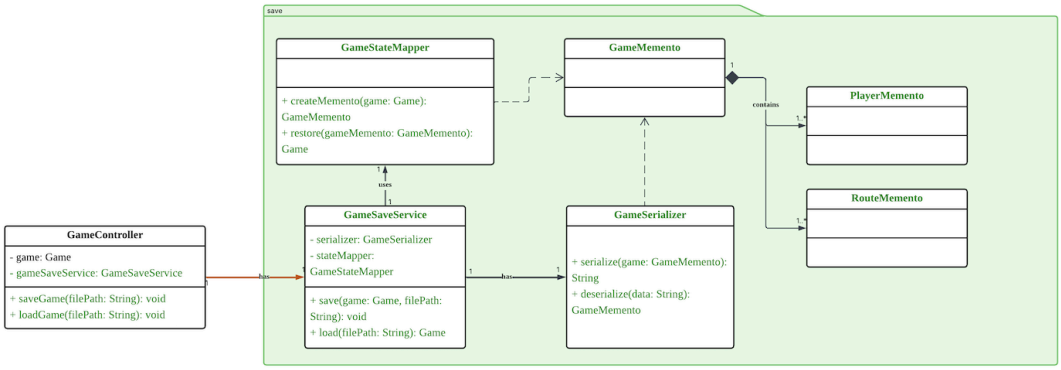


Figure 3: Save/Load sub-UML class diagram

2.3 Player Abilities

This self-defined extension is introduced to provide players with unique strategic advantages. Each player is able to choose a special ability that encourages different play styles. Two abilities were introduced:

- Ferry Master: rewards players who frequently claim ferry routes by allowing them to get a refund of 1 locomotive card each time they use locomotives to claim ferry routes. This effect may be used up to 2 times.
- Colour Master: rewards 2 scores to players for claiming different colour routes and grants an additional bonus when 4 unique route colors have been claimed by the player.

These abilities add new strategic considerations to encourage the players in making gameplay decision-making, adapting their route selection and card management.

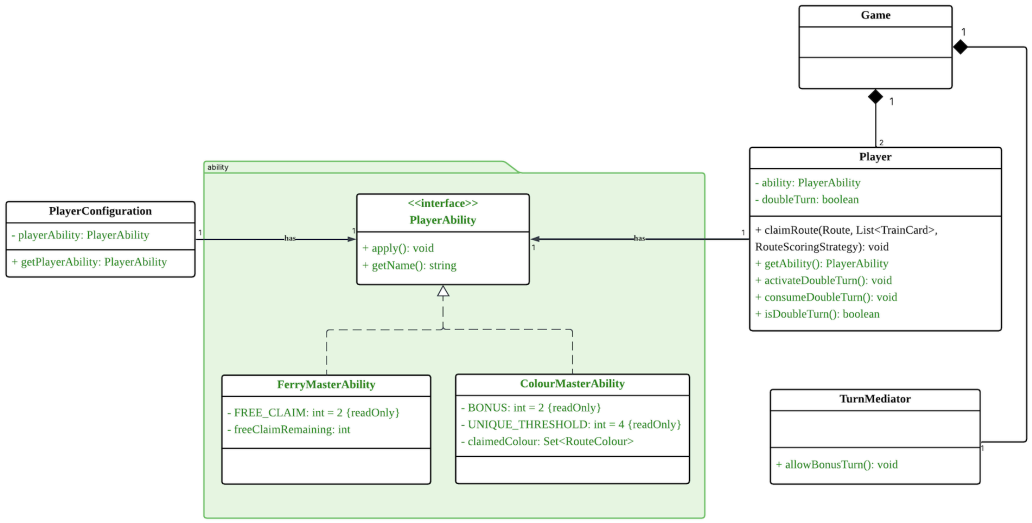


Figure 4: Player Ability sub-UML class diagram

2.4 Updated UML Class Diagram

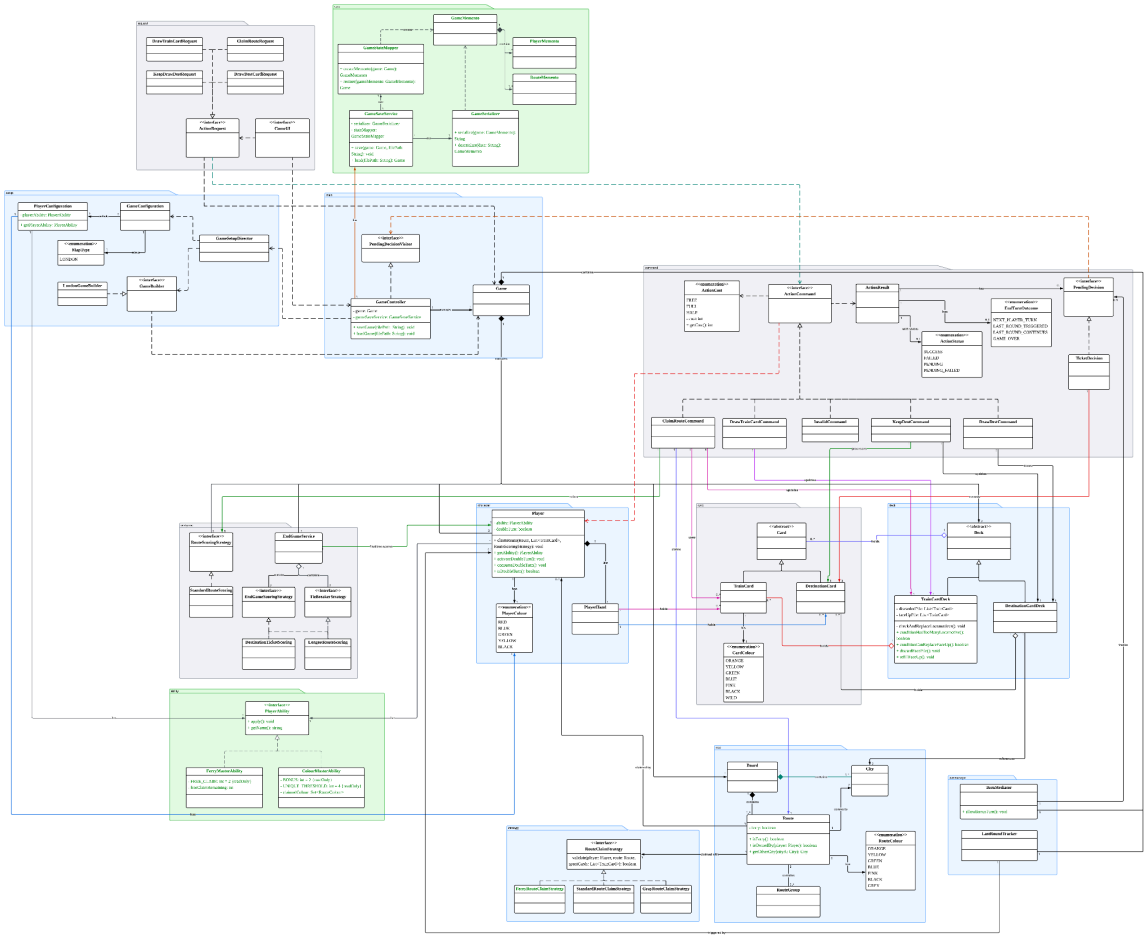


Figure 5: UML Class Diagram Final Version

2.3 Differences Between UML Diagram and Source Code

The source code contains some helper classes that are not shown in the UML class diagram, including PlayerStateMapper, RouteStateMapper, DeckStateMapper, and LastRoundStateMapper and AbilityFactory. These classes were added to delegate the responsibilities of GameStateMapper, who acts as a central coordinator for converting runtime game objects and memento objects. This delegation improves cohesion and maintainability of the GameStateMapper class. However, the behaviour shown in the diagram still presents the save/load system at a higher architectural level because GameStateMapper remains the main entry point for creating and restoring GameMemento objects.

3 Antipatterns and Refactoring

1. Duplicate Code in TrainCardDeck

Duplicate Code appears in two methods of TrainCardDeck, which is tryRefillFaceUpPile() and checkAndReplaceLocomotive(). Both methods use the same loop logic to refill face up piles. This is an example of the Duplicate Code antipattern and violates DRY principle.

Having these two duplications creates a maintenance problem because if any modification were made to the refill behaviour, it had to be updated in both locations. This increases the possibility of inconsistent behaviour and defects when one implementation is modified but the other one is overlooked. Moreover, it also reduces readability of the code as the same behaviour is scattered in two methods.

Hence, I applied the Extract Method refactoring technique. The duplicated refill logic was extracted into a dedicated helper method, refillEmptyFaceUpSlots(). Both tryRefillFaceUpPile() and checkAndReplaceLocomotive() now delegate the responsibility of replenishing face-up cards to this shared method rather than maintaining their own implementations.

This improves the codebase by removing duplicated refill logic and thus improves maintainability of the codebase. This is because the risk of inconsistent implementation has reduced, as there is only one place to change. Thus, all codes that require this method can behave correctly and consistently.

3.1.1 Before (Duplicated in two methods)

```
private void checkAndReplaceLocomotive() {  2 usages   ZiWeiNg +1

        for (int i = 0; i < FACE_UP_SIZE; i++) {
            if (faceUpPile.get(i) == null && !isInDeckEmpty()) {
                faceUpPile.set(i, draw());
            }
        }
    }
}
```

```
private void tryRefillFaceUpPile() {  2 usages  jhew0010 +1

    tryRefillInDeckPile();

    for (int i = 0; i < FACE_UP_SIZE; i++) {
        if (faceUpPile.get(i) == null && !isInDeckEmpty()) {
            faceUpPile.set(i, draw());
        }
    }

    // Ensure no have more than 3 face up locomotive cards
    checkAndReplaceLocomotive();
}
```

3.1.2 After (Extracted into one method)

```
private void refillEmptyFaceUpSlots() {  2 usages  jhew0010
    for (int i = 0; i < FACE_UP_SIZE; i++) {
        if (faceUpPile.get(i) == null && !isInDeckEmpty()) {
            faceUpPile.set(i, draw());
        }
    }
}
```

2. Feature Envy in Board

Feature Envy appears in the Board class. The Board contains route connectivity logic and ownership, such as: route.isClaimed(), route.getOwner(), route.getCityA(), route.getCityB() during DFS traversal and getConnectedCities() in Board. It repeatedly queried internal state from Route in order to determine route ownership and connectivity. This is an example of the Feature Envy antipattern.

This is problematic because the Board knew too much about the internal structure of Route. This designed tighter coupling because changes to Route would likely require modifications to Board. This also reduced encapsulation because route-related decisions were being made outside the Route class.

Hence, I applied the Move Method refactoring technique. I moved Route-specific behaviour into the Route class through methods getOtherCity(City city) and isOwnedBy(Player player).

This improves the codebase by reducing coupling between Board and Route. Moreover, it also increases encapsulation of Route internal logic, so the Board will no longer know the internal logic of Route, and will only query for results from Route.

3.2.1 Before (In Board)

```
private boolean dfs(Player player, City current,  2 usages  jhew0010 +1
                    City target, Set<City> visited) {
    if (current == target) {
        return true;
    }

    visited.add(current);

    for (Route route : routes) {

        if (!route.isClaimed() || route.getOwner() != player) {
            continue;
        }

        City next = getConnectedCity(route, current);

        if (next != null && !visited.contains(next)) {
            if (dfs(player, next, target, visited)) {
                return true;
            }
        }
    }

    return false;
}
```

```
private City getConnectedCity(Route route, City current) {  1 usage  jhew0010
    if (route.getCityA() == current) {
        return route.getCityB();
    }

    if (route.getCityB() == current) {
        return route.getCityA();
    }

    return null;
}
```

3.2.2 After (in Route)

```
212     /**
213      * Returns the city at the opposite end of this route from the given city.
214      *
215      * @param city one endpoint of the route
216      * @return the other endpoint city, or null if the given city
217      *     is not connected by this route
218      */
219     public City getOtherCity(City city) {  1 usage   jhew0010
220         if (cityA == city) return cityB;
221         if (cityB == city) return cityA;
222         return null;
223     }
224
225     /**
226      * Checks whether this route is owned by the specified player.
227      *
228      * @param player the player to check
229      * @return true if the route has been claimed by the player; false otherwise
230      */
231     public boolean isOwnedBy(Player player) {  2 usages   jhew0010
232         return isClaimed && owner == player;
233     }
```

DFS in Board

```
    private boolean dfs(Player player, City current,  2 usages   jhew0010 +1
                        City target, Set<City> visited) {
        if (current == target) {
            return true;
        }

        visited.add(current);

        for (Route route : routes) {
            if (!route.isOwnedBy(player)) {
                continue;
            }

            City next = route.getOtherCity(current);

            if (next != null && !visited.contains(next)) {
                if (dfs(player, next, target, visited)) {
                    return true;
                }
            }
        }

        return false;
    }
```

3. Long Method in EndGameService

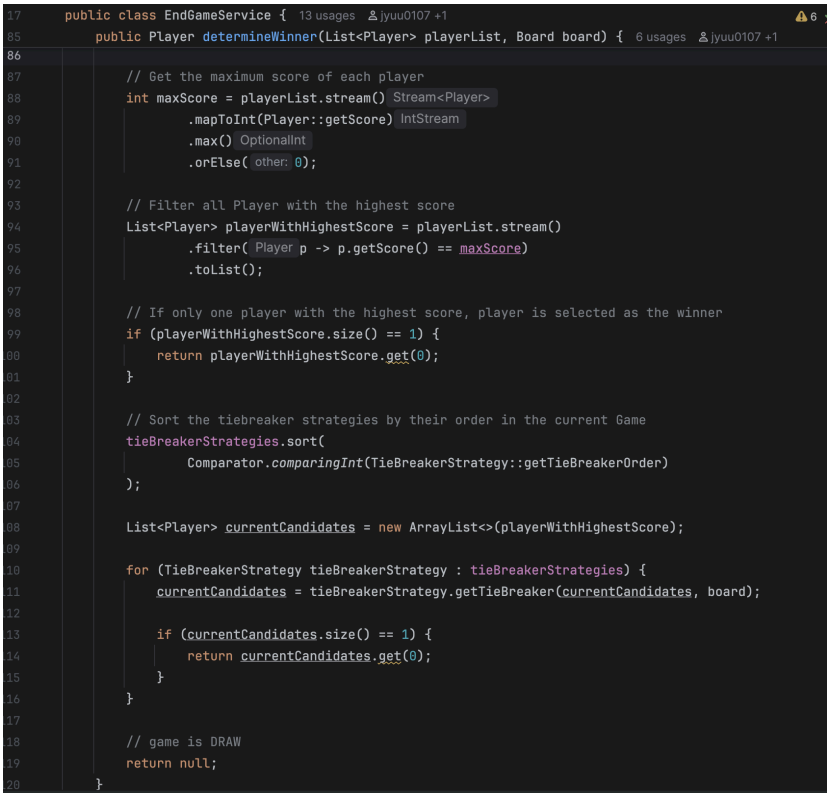
Long method is observed in EndGameService's determineWinner() method. It was responsible for finding the highest score, filtering players with that score, checking if there was only one winner, sorting tiebreaker strategies, applying each tiebreaker, and deciding whether the game ends in a draw. This is an example of the Long Method antipattern because one method handles too many responsibilities.

This is problematic as it makes the method harder to read, test, and modify. If the winner selection rules change later, the method may become even longer and more complex.

Hence, I applied the Extract Method refactoring technique by separating the logic into smaller helper methods, such as obtaining the highest score, filtering candidates, and applying tie-breakers.

This improves readability because determineWinner() now clearly describes the winner-selection process at a higher level of abstraction, making the overall workflow easier to understand. It also improves maintainability, as changes to a specific part of the winner determination logic can be made in a dedicated method without affecting unrelated functionality.

3.3.1 Before (Long Method with multiple responsibility)



```
public class EndGameService {  13 usages  jyuu0107 +1                                  6
    public Player determineWinner(List<Player> playerList, Board board) {  6 usages  jyuu0107 +1

        // Get the maximum score of each player
        int maxScore = playerList.stream()  Stream<Player>
                .mapToInt(Player::getScore)  IntStream
                .max()  OptionalInt
                .orElse( other: 0);

        // Filter all Player with the highest score
        List<Player> playerWithHighestScore = playerList.stream()
                .filter( Player p -> p.getScore() == maxScore)
                .toList();

        // If only one player with the highest score, player is selected as the winner
        if (playerWithHighestScore.size() == 1) {
            return playerWithHighestScore.get(0);
        }

        // Sort the tiebreaker strategies by their order in the current Game
        tieBreakerStrategies.sort(
                Comparator.comparingInt(TieBreakerStrategy::getTieBreakerOrder)
        );

        List<Player> currentCandidates = new ArrayList<>(playerWithHighestScore);

        for (TieBreakerStrategy tieBreakerStrategy : tieBreakerStrategies) {
            currentCandidates = tieBreakerStrategy.getTieBreaker(currentCandidates, board);

            if (currentCandidates.size() == 1) {
                return currentCandidates.get(0);
            }
        }

        // game is DRAW
        return null;
    }
```

3.3.2 After (Extracted into small methods)

```java
public Player determineWinner(List<Player> playerList, Board board) {  6 usages  jhew0010
    int highestScore = getHighestScore(playerList);
    List<Player> candidates = getPlayersWithScore(
            playerList,
            highestScore
    );

    if (candidates.size() == 1) {
        return candidates.get(0);
    }

    return applyTieBreakers(candidates, board);
}

/**
 * Returns the highest score among all players.
 *
 * @param playerList the players to check
 * @return the highest score, or {@code 0} if the list is empty
 */
private int getHighestScore(List<Player> playerList) {  1 usage  jhew0010 +2
    return playerList.stream() Stream<Player>
            .mapToInt(Player::getScore) IntStream
            .max() OptionalInt
            .orElse( other: 0);
}
```

```java
private List<Player> getPlayersWithScore(  1 usage  jhew0010
        List<Player> playerList,
        int score) {

    return playerList.stream()
            .filter( Player player -> player.getScore() == score)
            .toList();
}
```

```java
private Player applyTieBreakers(List<Player> candidates, Board board) {  1 usage
    sortTieBreakers();

    List<Player> currentCandidates = new ArrayList<>(candidates);

    for (TieBreakerStrategy tieBreakerStrategy : tieBreakerStrategies) {
        currentCandidates = tieBreakerStrategy.getTieBreaker(
                currentCandidates,
                board
        );

        if (currentCandidates.size() == 1) {
            return currentCandidates.get(0);
        }
    }

    return null;
}

/**
 * Sorts tie-breaker strategies by their configured order.
 */
private void sortTieBreakers() {  1 usage  jhew0010
    tieBreakerStrategies.sort(
            Comparator.comparingInt(TieBreakerStrategy::getTieBreakerOrder)
    );
}
```

4 Design Rationales

4.1 Ferry Route Extension

Alternative	**A:** Route class contains all route-claim validation logic through conditional statements	**B:** Strategy Pattern using RouteClaimStrategy
Advantage	- Simpler class structure with fewer classes - All validation logic can be located in a single place	- Encapsulates route validation algorithms separately from route data. - New route types can be added by introducing new strategy implementations without modifying existing classes - Improves cohesion and supports the OCP
Disadvantage	- Violates SRP, Route would be responsible for both route representation and validation - Every new route type would require modification of the Route class, reducing extensibility and increasing coupling	- Introduces additional classes and an extra level of indirection when validating route claims

Alternative B was chosen. This is because ferry routes differ primarily in their route-claim validation behaviour rather than their underlying route state. Since the RouteClaimStrategy abstraction had already been introduced in Sprint 2 to support different route-claiming rules, extending the existing Strategy Pattern was a natural design choice. The new FerryRouteClaimStrategy could therefore be added without modifying the Route class or existing strategy implementations, maintaining OCP. Although introducing a new strategy class increases the number of classes in the system, it significantly improves extensibility and maintainability by allowing future route types to be incorporated through additional strategy implementations rather than modifications to existing code.

4.2 Save and Load Extension

Alternative	**A:** State Pattern to represent different game states	**B:** Memento Pattern using GameMemento and specialised mapper classes
Advantage	- Clearly models different phases of the game - Can encapsulate behaviour that changes depending on the current phase	- Separates save/load concerns from gameplay concerns. - Save format can evolve independently from runtime classes.
Disadvantage	- Need another mechanism to serialize players, cards, routes, decks, and turn information - Introduce additional classes for each phase, increasing complexity	- Requires additional classes (GameMemento, mapper classes, serializer). - Increases implementation complexity.

Alternative B was chosen. The State Pattern was considered for modelling game phases such as setup, normal gameplay, pending decision, last round, and game over. However, it was not selected because the main design problem was not controlling phase-specific behaviour, but capturing and restoring complete game data. Using the State Pattern would still require a separate save/load mechanism to store cards, routes, players, scores, decks, turn cost, and last-round information.

Therefore, the Memento Pattern was more appropriate for this extension. It directly addresses the need to save and restore game state while keeping save/load concerns separate from gameplay classes. Although the State Pattern may be useful in a future refactoring if game-phase behaviour becomes more complex, it would be unnecessary and over-engineered for my current save/load requirement.

4.3 Player Ability Extension

Alternative	**A:** Store ability identifiers in Player and implement ability behaviour using conditional statements.	**B:** Strategy Pattern using PlayerAbility, ColourMasterAbility, and FerryMasterAbility.
Advantage	- Simple implementation with fewer classes - Ability logic remains inside the Player	- Encapsulates ability behaviour into specialised classes - New abilities can be introduced without modifying existing code

		- Improves cohesion, extensibility and testability.
Disadvantage	- Violates the OCP because every new ability requires modification of the Player - Ability logic becomes increasingly complex and reduces cohesion	- Additional interfaces and classes increase architectural complexity.

Alternative B was chosen. This is because player abilities introduce behaviour that varies independently from the core responsibilities of the Player class. Storing ability logic directly within Player would require the class to contain conditional statements for every ability in the game, causing it to become responsible for both player state management and ability execution**.**

The Strategy Pattern allows abilities to evolve independently while keeping the Player class focused on player state and gameplay interactions. Although this design introduces additional classes, the resulting improvement in extensibility, maintainability, and testability outweighs the increase in architectural complexity.

4.4 Quality Attributes and Trade-offs

The primary quality attribute considered throughout Sprint 3 was extensibility, with modifiability as a secondary concern. My Sprint 3 extensions were designed to support future gameplay

features and rule variations while minimizing the impact on existing classes. This allows new functionality to be incorporated without introducing widespread modifications throughout the system.

4.4.1 Quality Attribute Scenario

If a future extension introduces new player abilities or additional game-state information that must be saved by the game. The system should support these with minimal modification to existing gameplay classes and save/load infrastructure.

4.4.2 Tactics Applied

Tactic 1: Extensible Player Abilities

I introduced the PlayerAbility abstraction using the Strategy Pattern to satisfy this requirement. This allows new abilities to be introduced by implementing the PlayerAbility interface and assigning the new ability during player creation. Existing ability implementations and the Player class do not require modification when new abilities are added. This improves extensibility by encapsulating ability-specific behaviour into dedicated classes and reducing coupling between player state and ability logic. For example, a future ability such as a Route Master or Card Collector ability can be implemented as a new PlayerAbility class without requiring changes to the existing Player, ColourMasterAbility, or FerryMasterAbility classes. All ability logic can be implemented inside the new ability class.

Tactic 2: Extensible Save-State Representation

I introduced the Memento Pattern through GameMemento, PlayerMemento, RouteMemento, together with specialized mapper classes to satisfy this requirement. This separates save/load concerns from gameplay concerns and allows additional game-state information to be incorporated by extending the memento and mapper structures. Gameplay classes remain focused on gameplay behaviour rather than file handling and serialization responsibilities. This improves modifiability because changes to save state requirements can be implemented within the save/load subsystem without requiring large and wide modifications to the domain classes. For example, if a future extension introduces additional card types, the corresponding save-state information can be incorporated by extending the relevant memento and mapper classes. While the new feature itself may require changes to gameplay classes, the save/load subsystem can be updated independently without requiring many modifications to existing save/load logic.

4.4.3 Trade-offs

The primary trade-off is increased complexity. Compared to Sprint 2, the architecture introduces additional abstractions such as mementos, mappers, serializers and ability implementations. This increases the number of classes and relationships developers must understand and introduces additional layers of indirection. However, this complexity was considered acceptable because it significantly improves extensibility and modifiability. Future abilities and save-state components can be added with minimal changes to existing classes, reducing the likelihood of widespread modifications as the system evolves. Although the design introduces more classes, it improves separation of concerns and makes the system easier to extend and modify in future iterations.

5 Supplementary Documentation

6 AI Declaration

I also used generative AI (ChatGPT) to assist in:

- developing frontend UI code for advanced visual elements.
- Correcting grammar, rephrasing sentences, and improving rational explanations for writing clarity.

However, I reviewed, modified and integrated the generated advanced frontend UI code into the project as appropriate. All AI-generated text was carefully reviewed and further edited before inclusion. All final design decisions and implementation choices were made by me.